Universidad de las Fuerzas Armadas ESPE

Matriz – Sangolquí

Gestión de Proyectos de Software



Taller de SDD con Spec Kit — Cargorex

Docente: Ing. Jorge Geovanny Raura Ruiz

NRC: 36907

Antonio Adrián Revilla Anchapaxi

Carlos Orlando Hernandez Almeida

Sangolquí, 21 de octubre de 2026

Ficha del proyecto

Tabla 1

Datos de la primera iteración de Cargorex

Campo	Detalle
Asignatura	Gestión de Proyectos de Software
Institución	Universidad de las Fuerzas Armadas ESPE
Proyecto	Cargorex — Módulo de Distribución de Carga Horaria Docente
Iteración	Primera (01) — Prototipo de validación de horas de investigación
Metodología	Spec-Driven Development (SDD) con GitHub Spec Kit
Identificador de feature	001-research-hours-validation
Stack tecnológico	Python 3.11, FastAPI, SQLAlchemy, SQLite, Jinja2, HTML/CSS/JS, pytest
Fecha del informe	21 de octubre de 2026
Estado	Iteración completada y verificada (convergencia: converged)

Aviso: el prototipo utiliza datos ficticios de demostración. No se conecta a sistemas, APIs ni credenciales reales de la ESPE y no constituye una certificación institucional.

Introducción

El presente informe consolida el trabajo realizado durante el taller de Gestión de Proyectos de Software aplicando la metodología Spec-Driven Development (SDD) con la herramienta GitHub Spec Kit (GitHub, s. f.). El caso de estudio es Cargorex, módulo institucional de distribución de carga horaria docente, dentro del cual se identificó la necesidad de incorporar validaciones en la planificación de actividades. En particular, se tomó como ejemplo la exigencia de un proyecto de investigación aprobado para asignar horas de investigación.La metodología SDD se ejecutó en fases ordenadas y trazables: constitución → especificación → clarificación → planificación → tareas → implementación → análisis →

convergencia. Cada fase produjo un artefacto verificable, y este documento integra sus resultados reales sin reemplazar los originales.

Objetivo general

Desarrollar un prototipo web independiente que valide la asignación de horas de investigación de un docente contra proyectos de prueba con estado aprobado o no aprobado, aplicando validación en el servidor, persistencia local y pruebas automatizadas, y manteniendo trazabilidad completa entre requisitos, tareas, implementación y pruebas conforme a la constitución del proyecto.

Alcance de la iteración

Incluido: formulario web, selección de docente y proyecto ficticios, registro de horas enteras positivas, validación en el backend, mensajes de rechazo, almacenamiento en SQLite, consulta de registros y pruebas con pytest.

Excluido (fuera de alcance): autenticación institucional real, integración con APIs de la ESPE, aplicación de límites normativos de horas, aprobaciones departamentales, distribución completa de la carga horaria, reportería institucional, seguimiento por parciales y conexión con módulos de investigación reales.

Constitución del proyecto

Fuente: .specify/memory/constitution.md (versión 1.0.1, ratificada el 21 de octubre de 2026). Define diez principios de obligado cumplimiento:

Tabla 2

Principios de la constitución del proyecto

N.º	Principio	Síntesis
I	Desarrollo guiado por especificaciones	Requisitos, criterios, plan y tareas antes de codificar.

N.º	Principio	Síntesis
II	Trazabilidad completa	IDs RF-XXX y T###; cadena requisito → tarea → código → prueba.
III	Arquitectura modular	Separación presentación / negocio / datos (MVC adaptado a FastAPI).
IV	Validación en el servidor	Toda regla se valida en backend antes de persistir.
V	Calidad y pruebas	pytest con casos positivo, negativo y límite.
VI	Seguridad básica	Entradas saneadas, SQL parametrizado, sin credenciales en código.
VII	Simplicidad	Solo lo necesario para una primera iteración (YAGNI).
VIII	Requisitos verificables	No inventar normativa; marcar supuestos provisionales.
IX	Gestión de cambios	Todo cambio se propaga a spec, tareas y pruebas.
X	Documentación	Artefactos académicos en español; código en inglés.

La versión pasó de 1.0.0 a 1.0.1 (PATCH) durante la fase de análisis, para alinear la notación del identificador de tarea (T###) con la plantilla de Spec Kit. Sin cambios de principio ni de alcance.

Especificación de requisitos

Fuente: specs/001-research-hours-validation/spec.md. Incluye 3 historias de usuario priorizadas, 11 requisitos funcionales, 7 criterios de aceptación, casos límite y 7 criterios de éxito.

Historias de usuario: US1 (P1) registrar/validar asignación; US2 (P2) consultar asignaciones; US3 (P3) catálogo y estado de proyectos.

Requisitos funcionales

Tabla 3

Requisitos funcionales de la iteración

Req.	Descripción
RF-01	Seleccionar un docente ficticio del catálogo de prueba.
RF-02	Consultar y seleccionar proyectos de investigación de prueba.
RF-03	Mostrar el estado de cada proyecto (aprobado/pendiente/rechazado).
RF-04	Ingresar una cantidad positiva de horas (entero > 0).
RF-05	Verificar que el proyecto seleccionado exista.
RF-06	Permitir el registro solo si el proyecto está aprobado.
RF-07	Mostrar mensaje explicativo cuando el proyecto no esté aprobado.
RF-08	Rechazar horas ≤ 0, no enteras, no numéricas y entradas inválidas.
RF-09	Almacenar las asignaciones válidas.
RF-10	Consultar las asignaciones registradas.
RF-11	Impedir el almacenamiento de solicitudes inválidas aun por API directa.

Clarificaciones técnicas (sesión del 21 de octubre de 2026): horas como entero positivo; rechazos con HTTP 422 (entrada inválida) y HTTP 400 (regla de negocio), con JSON de motivo. Las cuestiones normativas (unidad/límite de horas, catálogo oficial de estados, duplicados) permanecen como dependencias pendientes de validación.

Planificación técnica

Fuente: specs/001-research-hours-validation/plan.md y artefactos de diseño asociados (research.md, data-model.md, contracts/openapi.yaml, quickstart.md). Para el stack se consultó la documentación oficial de FastAPI (s. f.), Pydantic (s. f.), SQLAlchemy (s. f.) y pytest (s. f.).

- Arquitectura: MVC adaptado a FastAPI con capa de servicio reutilizable.

- Modelo de datos: Teacher, ResearchProject (con enum ProjectStatus), ResearchAssignment.
- Contrato de API: OpenAPI 3.0.3 con endpoints de catálogo y de asignaciones (201/400/422).
- Decisiones clave (research.md): validación en dos niveles (Pydantic → 422; servicio → 400); horas entero > 0 sin límite máximo (pendiente); SQLite + SQLAlchemy; siembra idempotente.
- Verificación de constitución en el plan: 10/10 principios satisfechos; sin violaciones.

Tareas de implementación

Fuente: specs/001-research-hours-validation/tasks.md. 42 tareas (T001–T042) organizadas en seis fases, con etiquetas de historia, referencias a requisitos y una matriz de trazabilidad RF → Tareas → Pruebas.

Tabla 4

Fases y tareas de implementación

Fase	Tareas	Contenido
1. Preparación	T001–T006	Estructura, dependencias, .gitignore, configuración, servidor.
2. Modelo de datos	T007–T012	Entidades, SQLite, esquemas, siembra.
3. Backend	T013–T021	Validaciones, registro, consulta, manejo de errores.
4. Frontend	T022–T029	Formulario, selectores, estado, mensajes, tabla, estilos.
5. Pruebas	T030–T036	Configuración aislada, casos válidos/inválidos, API, persistencia.
6. Documentación	T037–T042	README, instalación, ejecución, evidencias, resultados.

Estado: 42/42 tareas completadas (100 %).

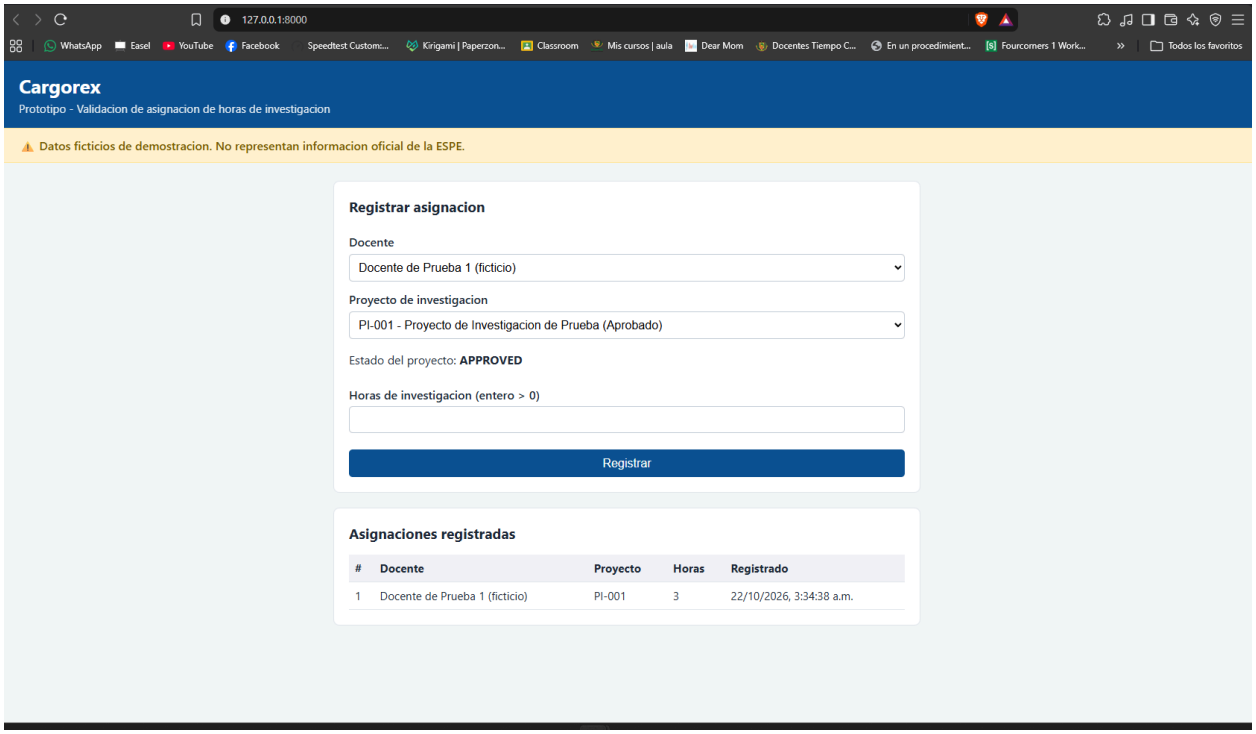
Implementación y evidencias

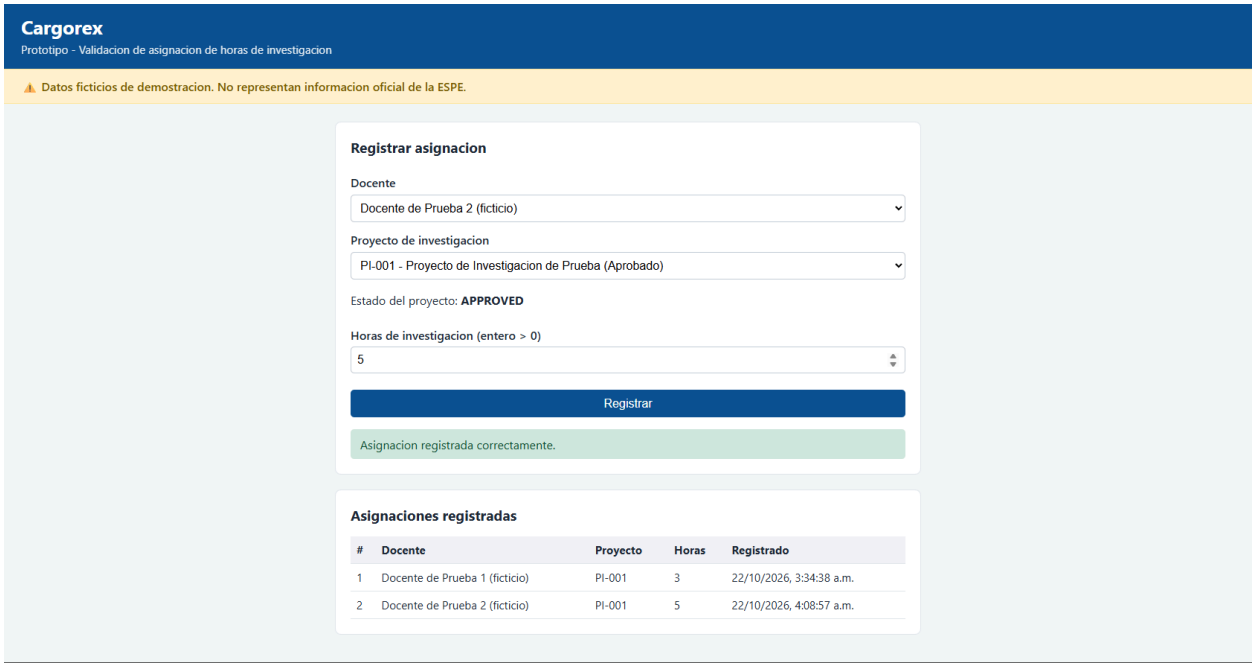
Fuentes: código en app/ y tests/; evidencia en specs/001-research-hours-validation/implementation.md y su carpeta evidence/.

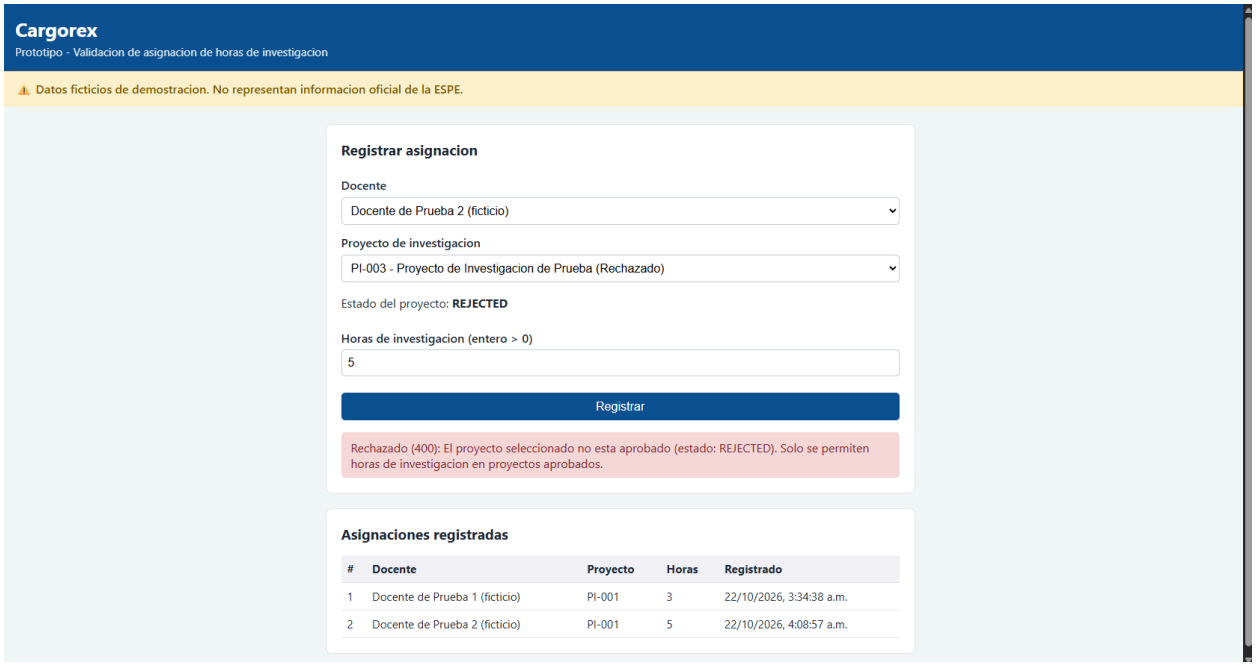
Resumen de implementación: los 11 requisitos se implementaron sobre la arquitectura planificada. La regla de negocio reside en app/services/assignment_service.py y se ejecuta en el servidor, independiente del cliente (RF-11). Un defecto real detectado durante la ejecución (cambio de firma de TemplateResponse en Starlette 1.9.0 que producía HTTP 500 en GET /) fue corregido en app/routers/pages.py y reverificado.

Verificación funcional (smoke test vía PowerShell, evidence/run-evidence.md): GET / → 200 con aviso de datos ficticios; catálogos con 3 docentes y estados APPROVED/PENDING/REJECTED; POST aprobado → 201; POST pendiente → 400; POST horas=0 → 422; consulta → 1 registro (solo la asignación válida).

Espacios para capturas de ejecución


127.0.0.1:8000
WhatsApp
Easel
YouTube
Facebook
Speedtest Custom...
Kirigami | Paperzon...
Classroom
Mis cursos | aula
Dear Mom
Docentes Tiempo C...
En un procedimient...
Fourcorners 1 Work...
Todos los favoritos
Cargorex
Prototipo - Validacion de asignacion de horas de investigacion
Datos ficticios de demostracion. No representan informacion oficial de la ESPE.
Registrar asignacion
Docente
Docente de Prueba 1 (ficticio)
Proyecto de investigacion
PI-001 - Proyecto de Investigacion de Prueba (Aprobado)
Estado del proyecto: APPROVED
Horas de investigacion (entero > 0)
Registrar
Asignaciones registradas
Docente Proyecto Horas Registrado
1 Docente de Prueba 1 (ficticio) PI-001 3 22/10/2026, 3:34:38 a.m.


Cargorex
Prototipo - Validacion de asignacion de horas de investigacion
Datos ficticios de demostracion. No representan informacion oficial de la ESPE.
Registrar asignacion
Docente
Docente de Prueba 2 (ficticio)
Proyecto de investigacion
PI-001 - Proyecto de Investigacion de Prueba (Aprobado)
Estado del proyecto: APPROVED
Horas de investigacion (entero > 0)
5
Registrar
Asignacion registrada correctamente.
Asignaciones registradas
Docente Proyecto Horas Registrado
1 Docente de Prueba 1 (ficticio) PI-001 3 22/10/2026, 3:34:38 a.m.
2 Docente de Prueba 2 (ficticio) PI-001 5 22/10/2026, 4:08:57 a.m.



Auditoría de cobertura

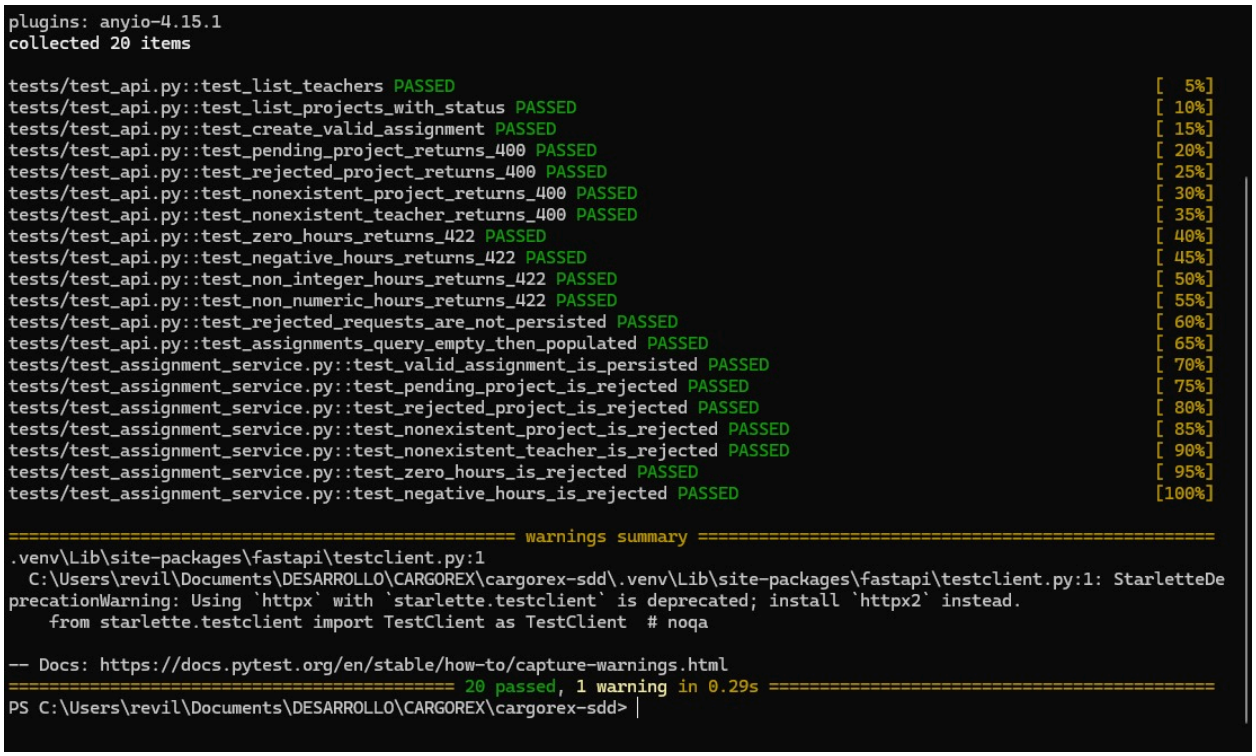
Fuente: specs/001-research-hours-validation/coverage.md. Resultados verificables consignados en el informe de la iteración:

Tabla 5

Resultados de la auditoría de cobertura

Métrica	**Valor**
Requisitos totales	11
Requisitos implementados	11
Requisitos verificados por prueba	11
Cobertura de requisitos	(11/11) × 100 = 100 %
Tareas totales / completadas	42 / 42 (100 %)
Pruebas ejecutadas (pytest -v)	20
Aprobadas / Fallidas / Omitidas	20 / 0 / 0
Resultado de convergencia	converged (sin brechas; tasks.md sin cambios)

Evidencia de pruebas



La advertencia es interna de starlette.testclient (ajena al código del prototipo). Las pruebas se ejecutan sobre una base de datos SQLite en memoria aislada.

Nota metodológica: la presentación visual en la interfaz (mensajes y estado) se verificó mediante smoke test manual, no por prueba automatizada de UI; la lógica de backend subyacente sí está cubierta automáticamente.

Conclusiones

1. La primera iteración es funcional, verificable y coherente con SDD: 11/11 requisitos implementados y probados, 42/42 tareas completadas y 20/20 pruebas en verde.
2. Se cumplió la validación en el servidor (Principio IV), incluso ante solicitudes enviadas directamente a la API, impidiendo la persistencia de datos inválidos.
3. Se mantuvo trazabilidad completa de extremo a extremo (requisito → tarea → código → prueba), preservando los identificadores RF-01…RF-11 y T001…T042.

4. La disciplina SDD permitió detectar y corregir inconsistencias (fase de análisis) y un defecto real de ejecución, manteniendo los artefactos consistentes entre sí.
5. Se respetó el alcance acotado y se evitó inventar normativa, registrando explícitamente las dependencias institucionales pendientes.

Recomendaciones

1. Realizar el levantamiento funcional definitivo con la Unidad de Docencia y UTIC para confirmar límites y unidad de horas, catálogo oficial de estados y reglas de acumulación/duplicados.
2. Adjuntar las capturas de ejecución señaladas en la sección 9.
3. Incorporar pruebas automatizadas de interfaz (end-to-end de navegador).
4. Diseñar la capa de integración hacia servicios institucionales reales cuando existan accesos y contratos.
5. Mantener la propagación de cambios spec → tareas → pruebas → código (Principio IX) en futuras iteraciones.

Anexos

Los documentos originales no se reemplazan ni eliminan; este informe solo los consolida.

Anexo A — Artefactos SDD

Tabla 6

Artefactos de Spec-Driven Development

Artefacto	Ruta
Constitución	.specify/memory/constitution.md
Especificación	specs/001-research-hours-validation/spec.md
Clarificaciones	Sección Clarifications dentro de spec.md
Plan técnico	specs/001-research-hours-validation/plan.md
Investigación (Fase 0)	specs/001-research-hours-validation/research.md
Modelo de datos	specs/001-research-hours-validation/data-model.md
Contrato de API	specs/001-research-hours-validation/contracts/openapi.yaml
Guía de inicio	specs/001-research-hours-validation/quickstart.md
Tareas	specs/001-research-hours-validation/tasks.md
Checklist de calidad	specs/001-research-hours-validation/checklists/requirements.md
Evidencia de implementación	specs/001-research-hours-validation/implementation.md
Auditoría de cobertura	specs/001-research-hours-validation/coverage.md
Resultados de pruebas	specs/001-research-hours-validation/evidence/test-results.md
Evidencia de ejecución	specs/001-research-hours-validation/evidence/run-evidence.md

Anexo B — Código fuente

Tabla 7

Componentes del código fuente

Componente	Ruta
Entrada / servidor	app/main.py
Configuración	app/config.py
Base de datos	app/database.py
Modelos ORM	app/models.py
Esquemas Pydantic	app/schemas.py
Siembra de datos	app/seed.py
Servicio de negocio	app/services/assignment_service.py
Rutas de página	app/routers/pages.py
Rutas de API	app/routers/api.py
Vista	app/templates/index.html
Estilos / script	app/static/css/styles.css; app/static/js/app.js
Pruebas	tests/conftest.py; tests/test_api.py; tests/test_assignment_service.py
Dependencias / guía	requirements.txt; README.md

Anexo C — Comandos de ejecución en Windows PowerShell

```
python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install -r requirements.txt

uvicorn app.main:app --reload   # App en http://127.0.0.1:8000

pytest -v   # Pruebas automatizadas
```